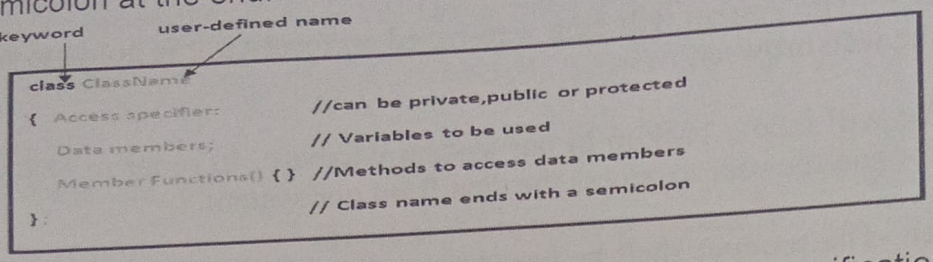
Defining Class and Declaring Objects

A class is defined in C++ using keyword class followed by the name of class. The body of class is defined inside the curly brackets and terminated by a semicolon at the end.



Declaring Objects: When a class is defined, only the specification for the object is defined; no memory or storage is allocated. To use the data and access functions defined in the class, you need to create objects.

Syntax:

```
ClassName ObjectName;
```

Accessing data members and member functions: The data members and member functions of class can be accessed using the dot('.') operator with the object. For example if the name of object is *obj* and you want to access the member function with the name *printName()* then you will have to write *obj.printName()* .

Accessing Data Members

The public data members are also accessed in the same way given however the private data members are not allowed to be accessed directly by the object. Accessing a data member depends solely on the access control of that data member.

This access control is given by Access modifiers in C++. There are three access modifiers : **public, private and protected**.

```
// C++ program to demonstrate
// accessing of data members

#include <iostream.h>
using namespace std;
class Geeks
{
    // Access specifier
    public:

    // Data Members
    string geekname;

    // Member Functions()
    void printname()
    {
       cout << "Geekname is: " <<
geekname;
    }
};

int main() {

    // Declare an object of class geeks
    Geeks obj1;
```

```
    // accessing data member
    obj1.geekname = "Abhi";

    // accessing member function
    obj1.printname();
    return 0;
}
```

Output:

```
Geekname is: Abhi
```

Member Functions in Classes

There are 2 ways to define a member function:

- Inside class definition
- Outside class definition

To define a member function outside the class definition we have to use the scope resolution :: operator along with class name and function name.

```
// C++ program to demonstrate function
// declaration outside class

#include <iostream.h>
using namespace std;
class Geeks
{
    public:
    string geekname;
    int id;

    // printname is not defined inside
class definition
    void printname();

    // printid is defined inside class
definition
    void printid()
    {
        cout << "Geek id is: " << id;
    }
};

// Definition of printname using scope
resolution operator ::
void Geeks::printname()
{
    cout << "Geekname is: " << geekname;
}
int main() {

    Geeks obj1;
    obj1.geekname = "xyz";
    obj1.id=15;

    // call printname()
    obj1.printname();
    cout << endl;
```

```
    // call printid()
    obj1.printid();
    return 0;
}
```

Output:

```
Geekname is: xyz
Geek id is: 15
```

Note that all the member functions defined inside the class definition are by default **inline**, but you can also make any non-class function inline by using keyword inline with them. Inline functions are actual functions, which are copied everywhere during compilation, like pre-processor macro, so the overhead of function calling is reduced.
Note: Declaring a friend function is a way to give private access to a non-member function.

CONSTRUCTORS

Constructors are special class members which are called by the compiler every time an object of that class is instantiated. Constructors have the same name as the class and may be defined inside or outside the class definition.
There are 3 types of constructors:

- Default constructors
- Parameterized constructors
- Copy constructors

```
// C++ program to demonstrate
constructors
 #include <bits/stdc++.h>
using namespace std;
class Geeks
{
    public:
    int id;

    //Default Constructor
    Geeks()
    {
        cout << "Default Constructor
called" << endl;
        id=-1;
    }
         //Parameterized Constructor
    Geeks(int x)
    {
        cout << "Parameterized
Constructor called" << endl;
        id=x;
    }
};
int main() {
         // obj1 will call Default
Constructor
    Geeks obj1;
    cout << "Geek id is: " <<obj1.id <<
endl;
         // obj1 will call Parameterized
Constructor
    Geeks obj2(21);
    cout << "Geek id is: " <<obj2.id <<
endl;
    return 0;
```

```cpp
// C++ program to demonstrate the use of default constructor

#include <iostream>
using namespace std;

// declare a class
class  Wall {
  private:
    double length;

  public:
    // default constructor to initialize variable
    Wall() {
      length = 5.5;
      cout << "Creating a wall." << endl;
      cout << "Length = " << length << endl;
    }
};

int main() {
  Wall wall1;
  return 0;
}
```

Output: Creating a Wall
Length = 5.5

Example 2: C++ Parameterized Constructor

```cpp
// C++ program to calculate the area of a wall

#include <iostream>
using namespace std;

// declare a class
class Wall {
  private:
    double length;
    double height;
```

```
  public:
    // parameterized constructor to initialize variables
    Wall(double len, double hgt) {
      length = len;
      height = hgt;
    }

    double calculateArea() {
      return length * height;
    }
};

int main() {
  // create object and initialize data members
  Wall wall1(10.5, 8.6);
  Wall wall2(8.5, 6.3);

  cout << "Area of Wall 1: " << wall1.calculateArea() << endl;
  cout << "Area of Wall 2: " << wall2.calculateArea();

  return 0;
}
```

Output

```
Area of Wall 1: 90.3
Area of Wall 2: 53.55
```

Example 3: C++ Copy Constructor

```
#include <iostream>
using namespace std;

// declare a class
class Wall {
  private:
    double length;
    double height;

  public:
```

```
    // initialize variables with parameterized constructor
    Wall(double len, double hgt) {
        length = len;
        height = hgt;
    }

    // copy constructor with a Wall object as parameter
    // copies data of the obj parameter
    Wall(Wall &obj) {
        length = obj.length;
        height = obj.height;
    }

    double calculateArea() {
        return length * height;
    }
};

int main() {
    // create an object of Wall class
    Wall wall1(10.5, 8.6);

    // copy contents of wall1 to wall2
    Wall wall2 = wall1;

    // print areas of wall1 and wall2
    cout << "Area of Wall 1: " << wall1.calculateArea() << endl;
    cout << "Area of Wall 2: " << wall2.calculateArea();

    return 0;
}
```

Output

```
Area of Wall 1: 90.3
Area of Wall 2: 90.3
```

In this program, we have used a copy constructor to copy the contents of one object of the `Wall` class to another. The code of the copy constructor is:

```
Wall(Wall &obj) {
  length = obj.length;
  height = obj.height;
}
```

Notice that the parameter of this constructor has the address of an object of the `Wall` class.

We then assign the values of the variables of the `obj` object to the corresponding variables of the object calling the copy constructor. This is how the contents of the object are copied.

In `main()`, we then create two objects `wall1` and `wall2` and then copy the contents of `wall1` to `wall2`:

```
// copy contents of wall1 to wall2
Wall wall2 = wall1;
```

Here, the `wall2` object calls its copy constructor by passing the address of the `wall1` object as its argument i.e. `&obj = &wall1`.

// Implicit copy constructor

```
#include<iostream>
using namespace std;
class Sample
{          int id;
    public:
    void init(int x)
    {
        id=x;
    }
    void display()
    {
        cout<<endl<<"ID="<<id;
    }
};
int main()
{
    Sample obj1;
    obj1.init(10);
    obj1.display();
    Sample obj2(obj1); //or obj2=obj1;
    obj2.display();
    return 0;}
```

copy constructor takes a reference to an object of the same class as an argument.

/ Example: Explicit copy constructor

```
#include <iostream>
using namespace std;
class Sample
{
    int id;
    public:
    void init(int x)
    {
        id=x;
    }
    Sample(){}  //default constructor with empty body

    Sample(Sample &t)   //copy constructor
    {
        id=t.id;
    }
    void display()
    {
        cout<<endl<<"ID="<<id;
    }
};
int main()
{    Sample obj1;
    obj1.init(10);
    obj1.display();
    Sample obj2(obj1); //or obj2=obj1;     copy constructor called
    obj2.display();
    return 0;}
```

Output

```
ID=10
ID=10
```

Interfaces in C++ (Abstract Classes)

Abstract classes are the way to achieve abstraction in C++. Abstraction in C++ is the process to hide the internal details and showing functionality only. Abstraction can be achieved by two ways:

1. **Abstract class**
2. **Interface**

Abstract class and interface both can have abstract methods which are necessary for abstraction.

C++ Abstract class

In C++ class is made abstract by declaring at least one of its functions as <>strong>pure virtual function. A pure virtual function is specified by placing "= 0" in its declaration. Its implementation must be provided by derived classes. We cannot create objects of an abstract class. However we can derive classes from them and use their data members and member functions (except pure virtual functions).

Let's see an example of abstract class in C++ which has one abstract method draw(). Its implementation is provided by derived classes: Rectangle and Circle. Both classes have different implementation.

```
#include <iostream>
using namespace std;
class Shape
{
  public:
  virtual void draw()=0;
};
class Rectangle : Shape
{
  public:
```

```
10.         void draw()
11.         {
12.             cout < <"drawing rectangle..." < <endl;
13.         }
14.     };
15.     class Circle : Shape
16.     {
17.         public:
18.         void draw()
19.         {
20.             cout <<"drawing circle..." < <endl;
21.         }
22.     };
23.     int main( ) {
24.         Rectangle rec;
25.         Circle cir;
26.         rec.draw();
27.         cir.draw();
28.         return 0;
29.     }
```

Output:

```
drawing rectangle...
drawing circle...
```

DESTRUCTOR

What is a destructor?

Destructor is an instance member function which is invoked automatically whenever an object is going to be destroyed. Meaning, a destructor is the last function that is going to be called before an object is destroyed.

- Destructor is also a special member function like constructor. Destructor destroys the class objects created by constructor.
- Destructor has the same name as their class name preceded by a tilde (~) symbol.
- It is not possible to define more than one destructor.
- The destructor is only one way to destroy the object create by constructor. Hence destructor can-not be overloaded.
- Destructor neither requires any argument nor returns any value.
- It is automatically called when object goes out of scope.
- Destructor release memory space occupied by the objects created by constructor.
- In destructor, objects are destroyed in the reverse of an object creation.

The thing is to be noted here, if the object is created by using new or the constructor uses new to allocate memory which resides in the heap memory or the free store, the destructor should use delete to free the memory.

Syntax:

```
Syntax for defining the destructor within the class

~ <class-name>()

{

}

Syntax for defining the destructor outside the class

<class-name>: : ~ <class-name>()

{

}
```

// Example:

```
#include<iostream>
using namespace std;

class Test
{
   public:
        Test()
       {
```

```
            cout<<"\n Constructor executed";
        }
        ~Test()
           {
            cout<<"\n Destructor executed";
           }
};
main()
{
     Test t;

    return 0;
}
```

// Example:

```
#include<iostream>
using namespace std;
int count=0;
class Test
{
    public:
        Test()
        {
            count++;
            cout<<"\n No. of Object created:\t"<<count;
        }

        ~Test()
        {
            cout<<"\n No. of Object destroyed:\t"<<count;
            --count;
        }
};

main()
{
    Test t,t1,t2,t3;
    return 0;
}
```

Properties of Destructor:

- Destructor function is automatically invoked when the objects are destroyed.
- It cannot be declared static or const.
- The destructor does not have arguments.
- It has no return type not even void.
- An object of a class with a Destructor cannot become a member of the union.
- A destructor should be declared in the public section of the class.
- The programmer cannot access the address of destructor.

When is destructor called?

A destructor function is called automatically when the object goes out of scope:

(1) the function ends
(2) the program ends
(3) a block containing local variables ends
(4) a delete operator is called

How are destructors different from a normal member function?

Destructors have same name as the class preceded by a tilde (~)
Destructors don't take any argument and don't return anything

Can there be more than one destructor in a class?

No, there can only one destructor in a class with class name preceded by ~, no parameters and no return type.

When do we need to write a user-defined destructor?

If we do not write our own destructor in class, compiler creates a default destructor for us. The default destructor works fine unless we have dynamically allocated memory or pointer in class. When a class contains a pointer to memory allocated in class, we should write a destructor to release memory before the class instance is destroyed. This must be done to avoid memory leak.

Can a destructor be virtual?

Yes, In fact, it is always a good idea to make destructors virtual in base class when we have a virtual function.

Garbage collection

Garbage collection is a memory management technique. It is a separate automatic memory management method which is used in programming languages where manual memory management is not preferred or done. In the manual memory management method, the user is required to mention the memory which is in use and which can be deallocated, whereas the garbage collector collects the memory which is occupied by variables or objects which are no more in use in the program. Only memory will be managed by garbage collectors, other resources such as destructors, user interaction window or files will not be handled by the garbage collector.

Few languages need garbage collectors as part of the language for good efficiency. These languages are called as garbage-collected languages. For example, Java, C# and most of the scripting languages needs garbage collection as part of their functioning. Whereas languages such as C and C++ support manual memory management which works similar to the garbage collector. There are few languages that support both garbage collection and manually managed memory allocation/deallocation and in such cases, a separate heap of memory will be allocated to the garbage collector and manual memory.

Some of the bugs can be prevented when the garbage collection method is used. Such as:

- dangling pointer problem in which the memory pointed is already deallocated whereas the pointer still remains and points to different reassigned data or already deleted memory
- the problem which occurs when we try to delete or deallocate a memory second time which has already been deleted or reallocated to some other object
- removes problems or bugs associated with data structures and does the memory and data handling efficiently
- memory leaks or memory exhaustion problem can be avoided

Let's see a detailed understanding of manual memory management vs garbage collection, advantages, disadvantages and how it is implemented in C++.

Manual Memory Management

Dynamically allocated memory during run time from the heap needs to be released once we stop using that memory. Dynamically allocated memory takes memory from the heap, which is a free store of memory.

In C++ this memory allocation and deallocation are done manually using commands like new, delete. Using 'new' memory is allocated from the heap. After its usage, this memory must be cleared using the 'delete' command.

Nesting of Member functions

As we known that a member function of a class can be called only by an object of that class using a dot operator. However, there is an exception to this. A member function can be called by using its name inside another member function of the same class. This is known as nesting of member functions.

```
#include<iostream.h>
class set
{ int m, n;
public:
  void input(void);
  void display(void);
  int largest(void);
};
int set :: largest(void)
{
  if(m>=n)
    return(m);
  else
    return(n);
}
void Set :: input(void)
{
  cout << "Input values of m and n" << "\n";
  cin >> m >> n;
}
void set :: display(void)
{
cout << "Larget value =" << largest() << "\n" //calling
// member function
}
int main()
{ set A;
  A.input();
  A.display();
return 0;
```

Static Data Members :-

A data member of a class can be qualified as static. The properties of a static member variable are similar to that of a C static variable. A static member variable has certain special characteristics. These are:-

1) It is initialized to zero when the first object of its class is created. No other initialization is permitted.
2) Only one copy of that member is created for the entire class and is shared by all the objects of that class, no matter how many objects are created.
3) It is visible only within the class, but its lifetime is the entire program.

Static variables are normally used to maintain values common to the entire class. For e.g:- a static data member can be used as a counter that records the occurence of all the objects.

```
#include<iostream.h>
class item
{ static int count;
  int number;
public:
  void getdata(int a)
  { number = a;
    count++;
  }
  void getcount(void)
  { cout << "count:";
    cout << count << "\n";
  }};
int item::count; //definition of static data member
int main()
{ item a,b,c; // count is initialized to zero.
  a.getcount(); // display count
  b.getcount();
  c.getcount();
  a.getdata(100); //getting data into object a
  b.getdata(200); //  "   "   "   "  b
  c.getdata(300); //  "   "   "   "  c
  cout << "After reading data" << "\n";
  a.getcount(); // display count
  b.getcount();
  c.getcount();
  return 0;
}
```

OUTPUT

```
count:0
count:0
count:0
After reading data
count:3
count:3
count:3
```

⇒ Following statement in the program:-

```
int item :: count;  // definition of static data member.
```

Note that the type and scope of each static member variable must be defined outside the class definition. This is necessary because the static data members are stored separately rather than as a part of an object. Since they are associated with the class itself rather than with any class object; they are also known as class variables.

The ~~static~~ The static variable count is initialized to zero when the objects are created. The count is incremented whenever the data is read in to an object. Since the data is read into objects three times, the variable count is incremented 3 time. B'cos there is only one copy of count shared by all three objects, all the three output statements cause the value 3 to be displayed.

⇒ Static variables are like non-inline member functions as they are declared in a class and defined in the source file.

STATIC Member Function

A member function that is declared static has the following properties:

1) A static function can have access to only other static members (functions or variables) declared in the same class.
2) A static member function can be called using the class name (instead of its objects)

```
class-name :: function-name;
```

```
(*) static void showcount(void)
{ cout << " count : " << count << "\n";
}};
```

```
#include <iostream.h>
class test
{ int code;
  static int ~~code~~ count;
public:
  void setcode(void)
  { code = ++count;
  }
  void showcode(void)
  { cout << "object number : " << code << "\n";
  }
(*)
```

```
int test :: count; // definition of static data member
int main()
{ test t1, t2;
  t1.setcode();
  t2.setcode();
  test :: showcount(); // accessing static function
  test t3;
  t3.setcode();
  test :: showcount();
  t1.showcode();
  t2.showcode();
  t3.showcode();
  return 0;
}
```

Object Number : 2
Object Number : 3

Note :- code = ++count; is executed whenever constructor is invoked and the current value of count is assigned to code. Since each object has its own copy of code, the value contained in code represents a unique number of its object.

OBJECTS as function Arguments

An object may be used as a function argument. This can be done in two ways:

1) A copy of the entire object is passed to the function
2) only the address of the object is transfered to the function.

```
#include <iostream.h>
class time
{ int hours, minutes;
public:
void gettime(int h, int m)
{ hours = h; minutes = m; }
void puttime(void)
{ cout << hours << " hours and ";
  cout << minutes << " minutes" << "\n"; }
void sum(time, time); //declaration with objects as arguments.
};
void time :: sum(time t1, time t2) // t1, t2 are objects.
{
  minutes = t1.minutes + t2.minutes;
  hours = minutes/60;
  minutes = minutes % 60;
  hours = hours + t1.hours + t2.hours;
}
int main()
{
  time T1, T2, T3;
  T1.gettime(2, 45); //get T1
  T2.gettime(3, 30); //get T2
  T3.sum(T1, T2); // T3 = T1 + T2
  cout << "T1 = "; T1.puttime(); //display T1
  cout << "T2 = "; T2.puttime(); // " T2
  cout << "T3 = "; T3.puttime(); // " T3
  return 0;
}
```

(OUTPUT)
T1 = 2 hours and 45 minute
T2 = 3 hours and 30 minutes
T3 = 6 hours and 15 minu

Note :- Since the member function sum() is invoked by the objects T3, with the objects T1 and T2 as arguments, it can directly access the hours and minutes variables of T3. But the members of T1 & T2 can be accessed only by using the dot operator (like T1.hours and T1.minutes). Therefore, inside the function sum(), the variables hours and minutes refer to T3. T1.hours and T1.minutes refer to T1 and T2.hours and T2.minutes refer to T2.

Scope resolution operator

```
#include<iostream.h>
int m = 10;    //global m
int main()
{
 int m = 20;  // m redeclared, local to main
 {
  int k= m;
  int m = 30;  // m declared again
               // local to inner block
cout << "we are in inner block \n";
 cout << "k =" << k << "\n";
 cout << "m =" << m << "\n";
 cout << "::m =" << ::m << "\n";
 }
 cout << "\n We are in outer block \n";
  cout << "m = " << m << "\n";
  cout << "::m = " << ::m << "\n";
  return 0;
  }
```

Output:-

We are in inner block
k = 20
m = 30
::m = 10
We are in Outer block
m = 20
::m = 10

Memory Management Operators

C uses malloc() and calloc() functions to allocate memory dynamically at run time. Similarly it uses the function free() to free dynamically allocated memory. We use dynamic allocation techniques when it is not known in advance how much of memory space is needed. Although C++ supports these functions, it also defines two unary operators new and delete that perform the task of allocating and freeing the memory in a better and easier way. Since these operators manipulate memory on the free store, they are also known as free store operators.

An object can be created by using 'new', and destroyed by using 'delete', as and when required. A data object created inside a block with new, will remain in existence until it is explicitly destroyed by using 'delete'. Thus the life time of an object is directly under our control and is unrelated to the block structure of the program.

The new operator can be used to create objects of any type. It takes the following general form

pointer_variable = new datatype;

Here, pointer_variable is a pointer of type data-type. The 'new' operator allocates sufficient memory to hold a data object of type data-type and returns the address of the object. The datatype may be any valid data type. The pointer variable holds the address of the memory space allocated

```
                    int *p;   float *q;   - (1)
E.g :- p = new int;
       q = new float;                     - (2)
```

Here 'p' is a pointer of type int and 'q' is a pointer of type float.

We can also combine the above statements like :-

```
int *p = new int;
float *q = new float;
```

Subsequently the statements

```
*p = 25;
*q = 7.5;
```

assign 25 to the newly created 'int' object and 7.5 to the 'float' object.

We can also initialize the memory using the new operator.

ie pointer-variable = new datatype (value);

```
int *p = new int (25);
float *q = new float (7.5);
```

⇒ 'New' can be used to create a memory space for any data type including user defined types such as arrays, structures and classes.

general form for a 1 dimensional array is

pointer-variable = new data-type [size];

Here size specifies the number of elements in the array

int *p = new int [10]; ⇒ creates a memory space for an array of 10 integers. p[0] will refer to the first element p[1] to the second and so on.

For Multidimensional array:

array_ptr = new int [3] [5] [4];

Delete Operator:- When a data object is no longer needed, it is destroyed to release the memory space for reuse.

general form:-

```
delete pointer-variable;
delete p;
delete q;
```

If we want to free a dynamically allocated array then

delete [size] pointer-variable;

The size specifies the number of elements in the array to be freed.

dele [] p; will delete the entire array pointed to by p.

Subsequently the statements

```
*p = 25;
*q = 7.5;
```

assign 25 to the newly created 'int' object and 7.5 to the 'float' object.

We can also initialize the memory using the new operator.

ie pointer-variable = new datatype (value);

```
int *p = new int (25);
float *q = new float (7.5);
```

⇒ 'New' can be used to create a memory space for any data type including user defined types such as arrays, structures and classes.

general form for a 1 dimensional array is

pointer-variable = new data-type [size];

Here size specifies the number of elements in the array

```
int *p = new int [10];
```

creates a memory space for an array of 10 integers. p[0] will refer to the first element p[1] to the second and so on.

For multidimensional array:

```
array_ptr = new int [3][5][4];
```

Delete operator:- When a data object is no longer needed, it is destroyed to release the memory space for reuse.

general form:-

```
delete pointer-variable;
delete p;
delete q;
```

If we want to free a dynamically allocated array then

```
delete [size] pointer-variable;
```

The size specifies the number of elements in the array to be freed.

dele [] p; will delete the entire array pointed to by p.

If sufficient memory is not available for allocation, then 'new' returns a null pointer like malloc(). Therefore it is better to check for the pointer produced by 'new' before using it.

```
- - -
- - - -
  p = new int;
  if(!p)
{
cout << " allocation failed \n";
}
  - - -
  - - -
```

The new operator offers the following advantages over the function malloc().

1) It automatically computes the size of the data object. We need not use the operator sizeof.
2) It automatically returns the correct pointer type, so that there is no need to use a type cast.
3) It is possible to initialize the object while creating the memory space
4) Like any other operator, new and delete can be overloaded.

malloc Syntax :- ptr = (cast-type *) malloc(byte-size)
e.g. :- ptr = (int *) malloc(100 * sizeof(int));

```
#include <iostream.h>
int main()
int *p;
float *q;
  p = new int;
  q = new float;
*p = 45;
*q = 45.45f;
cout << *p << endl;
cout << *q << endl;
delete p;
delete q;
return 0;
}
```

(Example of new and delete operator in C++)